

CWSO: Next-Generation Effizienz & Hardware-Architektur

Diese Erweiterung des **Concurrent Workspace & Swarm Orchestrator (CWSO)** integriert modernste Forschungsergebnisse aus den Bereichen sparsame Inferenz, alternative Hardware-Architekturen und neue Modell-Paradigmen.

Ziel ist es, den CWSO von einem reinen Software-Orchestrator zu einem **Hardware- und Architektur-bewussten Hypervisor für Agenten-Schwärme** zu machen, der extreme Skalierung bei minimalem Ressourcenverbrauch ermöglicht.

1. Forschungsgrundlagen (Zusammenfassung)

Um die neuen Features zu begründen, stützt sich der CWSO auf folgende technologische Durchbrüche:

- **Sparse Models & Pruning:** Modelle nutzen "Sparsity" (Kargheit), bei der inaktive Parameter (Nullen) bei der Berechnung übersprungen werden. Gepaart mit 1-Bit/1.58-Bit Quantisierung (z.B. BitNet) sinkt der VRAM-Bedarf und die Inferenzzeit drastisch.
- **State Space Models (SSMs):** Architekturen wie Mamba oder Jamba, die eine lineare Skalierung der Kontextlänge (statt quadratisch wie bei Transformern) ermöglichen. Ideal für riesige Codebases.
- **Neuromorphes Computing:** Hardware (wie Intel Loihi), die biologische Gehirne simuliert (Spiking Neural Networks). Sie verbraucht fast null Energie, bis ein "Event" (Spike) eintritt.
- **Photonisches Computing & LPUs:** Optische Chips, die Matrixmultiplikationen mit Lichtgeschwindigkeit durchführen, und LPUs (Language Processing Units, z.B. Groq), die auf extrem schnelle, deterministische Inferenz für LLMs optimiert sind.

2. Neue CWSO-Features (Detaillierte Implementierung)

Basierend auf dieser Recherche fügen wir dem CWSO vier hochgradig detaillierte Kernfeatures hinzu:

Feature A: Heterogeneous Hardware Dispatcher (HHD)

Der CWSO delegiert Aufgaben nicht mehr einfach nur an "ein LLM", sondern analysiert die Aufgabe (Latenz vs. Kontext vs. Logik) und routet sie an das effizienteste Hardware- oder Architektur-Backend.

- **Implementierungsdetails (Go/Rust):**
 - **Profiling-Layer:** Jede eingehende Aufgabe erhält einen Tensor-Tag.
 - *Beispiel:* Ein simpler Syntax-Fix in einer Datei -> Routing an eine **LPU-API (Groq)** für millisekunden-schnelle Generierung.
 - *Beispiel:* Die Analyse einer 100k-Zeilen-Codebase -> Routing an ein **SSM (Mamba-basiertes Modell)** auf klassischer GPU-Hardware, um den gigantischen Kontext ohne OOM-Errors (Out of Memory) zu verarbeiten.
 - **Dispatcher-Logik:** Erweiterung des `dispatch_concurrent_jobs` Tools um den

- Parameter `hardware_target`: "lpu" | "gpu" | "photonic_sim" | "edge_npu".
- **Vorteil:** Maximale Kosteneffizienz und drastisch reduzierte Time-to-Merge.

Feature B: Ephemere Wasm-Micro-Agents (1.58-Bit Quantized)

Die größte Gefahr bei "Shadow Workspaces" ist die Latenz beim Starten von Docker-Containern für Sub-Agenten. Wir ersetzen schwere Container durch extrem quantisierte Modelle.

- **Implementierungsdetails (Go/Rust):**
 - Statt eines 8B-Parameter Modells im Container nutzt der CWSO **1.58-Bit quantisierte Modelle (BitNet-Architektur)**. Diese Modelle können vollständig im CPU-RAM gehalten werden.
 - **WebAssembly (Wasm) Sandboxing:** Sub-Agenten werden als Wasm-Module (via `wasmtime` in Rust oder `wazero` in Go) gestartet.
 - **Workflow:** Der CWSO nutzt *Unstructured Pruning*, um winzige, domänenspezifische Agenten on-the-fly zu erstellen (z.B. ein Agent, der *nur* React-Hooks versteht). Ein Wasm-Agent startet in **< 5 Millisekunden** (im Vergleich zu 2-3 Sekunden bei Docker).
 - **Ressourcen-Streaming:** Über die MCP SSE-Verbindung streamt der CWSO den RAM-Verbrauch und die Token-Generierung der Micro-Agents in Echtzeit an die Master-KI.

Feature C: Event-Driven "Spiking" AST-Monitore (Neuromorphes Prinzip)

Aktuelle Systeme pollen (fragen zyklisch ab), ob sich Dateien geändert haben. Das ist ineffizient. Wir adaptieren neuromorphe Konzepte (Spiking Neural Networks) für das Datei-System-Monitoring.

- **Implementierungsdetails (Go/Rust):**
 - Nutzung von **eBPF (Linux)** oder tiefen OS-Hooks, die nur bei Schreibzugriffen auf den AST (Abstract Syntax Tree) auslösen.
 - **Spike-Filter:** Wenn Sub-Agent A in seinem Shadow Workspace tippt, feuert ein "Spike". Ein lokales, extrem sparsames (Sparse) Mini-Modell evaluiert in Mikrosekunden, ob dieser Spike einen semantischen Konflikt mit Agent B auslösen könnte (z.B. Signaturänderung einer Funktion).
 - **Schlafmodus:** Solange keine relevanten semantischen Spikes auftreten, verbraucht der Monitoring-Thread *exakt null* CPU-Zyklen. Es gibt keine Polling-Loops mehr.
 - **MCP Tool-Erweiterung:** Ein neues Tool `subscribe_ast_spikes(path: string)`, das über SSE nur dann asynchrone Events sendet, wenn semantische Schwellenwerte überschritten werden.

Feature D: Semantic Sparse-Merging (SSM)

Der `merge_concurrent_results` Prozess profitiert von *Sparse Representation* des Codes.

- **Implementierungsdetails (Go/Rust):**
 - Wenn der CWSO zwei parallele Shadow Workspaces zusammenführt, wandelt er

- den Code nicht in Text, sondern in **Sparse AST-Tensoren** um.
- Unveränderte Code-Bereiche (die "Nullen" in der Sparse-Matrix) werden mathematisch komplett ignoriert.
- **Photonic-Readiness:** Die Diff-Berechnung des ASTs wird so vektorisiert, dass sie perspektivisch auf photonischen Co-Prozessoren (die extrem gut in Vektor-Multiplikationen sind) mit Lichtgeschwindigkeit und ohne Erhitzung berechnet werden kann. Bis photonische Hardware standard ist, wird SIMD (AVX-512) in Rust genutzt, um diese Vektoren zu verarbeiten.

3. Aktualisierte MCP Tool Definitionen

`dispatch_hardware_aware_job`

Ersetzt das klassische `dispatch_concurrent_jobs`

- **Inputs:**
 - `task_description` (string)
 - `context_size_estimate` (integer): Geschätzte Anzahl der Token.
 - `latency_requirement` (enum): `realtime` | `batch`
- **CWSO Interne Logik:** Wählt autonom zwischen LPU (für realtime & kleine Kontexte), SSM/Mamba (für riesige Kontexte) oder lokalen Wasm-Micro-Agents (für batch Code-Linting).
- **Outputs:** `job_id`, `assigned_hardware_profile`

`create_ephemeral_sparse_agent`

Ersetzt schwerfällige `Shadow Workspaces`

- **Inputs:**
 - `target_ast_node` (string): z.B. "Class: DatabaseConnector"
 - `quantization` (string): e.g., "1.58-bit"
- **Outputs:** * `wasm_agent_id`
 - Startzeit (sollte bei < 10ms liegen).

4. Architektonische Risiken & Mitigation

1. **Risiko:** Fehlende API-Standards für heterogene Hardware (z.B. neuromorphe APIs).
 - *Mitigation:* Aufbau einer Abstraktionsschicht (Hardware Abstraction Layer - HAL) in Rust. Hardware-Backends werden als Plugins geladen. Fällt ein Backend aus, wird transparent auf Standard-GPUs (CUDA/Metal) zurückgegriffen.
2. **Risiko:** Qualität von 1-Bit quantisierten Modellen für komplexe Programmieraufgaben.
 - *Mitigation:* Wasm-Micro-Agents werden *nur* für deterministische, kleine Tasks eingesetzt (z.B. Typisierung hinzufügen, Variablen umbenennen). Für komplexe Architektur-Entscheidungen triggert der CWSO automatisch ein "Dense Model" (z.B. GPT-4o oder Claude 3.5 Sonnet) auf GPU-Clustern.